

Proyecto No. 1

Máquina de Turing para Fibonacci

Mia Alejandra Fuentes Mérida - 23775
Angel Esteban Esquit Hernández - 23221

27 de febrero de 2026

1. Notación Asintótica O (Big-O)

La notación Big-O describe el límite superior del crecimiento de una función cuando el tamaño de la entrada tiende a infinito.

Formalmente:

$$f(n) \in O(g(n))$$

si existen constantes positivas c y n_0 tales que:

$$0 \leq f(n) \leq cg(n) \quad \forall n \geq n_0$$

Big-O permite modelar el crecimiento del número de pasos ejecutados por un algoritmo conforme aumenta el tamaño de la entrada.

2. Convenciones de Representación

2.1. Representación Unaria

Se utiliza representación unaria:

- $0 \rightarrow$ cadena vacía
- $1 \rightarrow 1$
- $2 \rightarrow 11$
- $n \rightarrow 1^n$

La entrada consiste en una cadena de n símbolos 1.

2.2. Alfabetos

Alfabeto de entrada:

$$\Sigma = \{1\}$$

Alfabeto de la cinta:

$$\Gamma = \{1, -, |, \#, n, x, a, A, b, B, c\}$$

Estos símbolos permiten implementar los acumuladores y marcadores necesarios para la recurrencia iterativa.

3. Definición Formal de la Máquina

La Máquina de Turing se define como la 7-tupla:

$$M = (Q, \Sigma, \Gamma, \delta, qStart, -, F)$$

Donde:

- Q es el conjunto finito de estados (30 estados).
- $\Sigma = \{1\}$
- $\Gamma = \{1, -, |, \#, n, x, a, A, b, B, c\}$
- δ es la función de transición
- $qStart$ es el estado inicial
- $F = \{qAccept\}$

La función de transición está definida en el archivo `fib_tm_real.json` y contiene 132 transiciones.

4. Idea General del Funcionamiento

Implementa la recurrencia:

$$F(n) = F(n-1) + F(n-2)$$

La estrategia consiste en:

1. Convertir la entrada 1^n en marcadores n .
2. Inicializar acumuladores:

$$a = F(0), \quad b = F(1)$$

3. Por cada símbolo n restante:

- Copiar a y b hacia un área temporal
- Construir $c = a + b$
- Reasignar:

$$a \leftarrow b$$

$$b \leftarrow c$$

4. Al finalizar, limpiar marcadores y dejar únicamente b en notación unaria.

Esto produce una simulación iterativa real de Fibonacci en una sola cinta.

5. Análisis Empírico

Se realizaron pruebas para:

$$n = 0, 1, 2, \dots, 18$$

Con límite de 50,000,000 pasos.

Para cada ejecución se registró:

- Pasos ejecutados
- Tiempo de CPU
- Estado final
- Valor leído en cinta

Se observó crecimiento acelerado conforme aumenta n .

5.1. Listado de Entradas de Prueba y Resultados

En la Tabla 1 se presentan los valores utilizados en el análisis experimental, junto con el número de pasos ejecutados, el tiempo de CPU medido y el estado final de la máquina.

n	Pasos	Tiempo (s)	Estado
0	11	0.000014	OK
1	53	0.000048	OK
2	130	0.000070	OK
3	262	0.000129	OK
4	512	0.000251	OK
5	1009	0.000509	OK
6	2077	0.001009	OK
7	4503	0.002173	OK
8	10284	0.005038	OK
9	24512	0.011779	OK
10	60330	0.029131	OK
11	151811	0.073541	OK
12	387647	0.185117	OK
13	999177	0.503977	OK
14	2590694	1.266678	OK
15	6742026	3.267365	OK
16	17585636	8.502928	OK
17	45934629	22.281731	OK
18	50000000	24.341875	ERROR

Cuadro 1: Resultados del análisis empírico para $0 \leq n \leq 18$.

6. Diagrama Simplificado de Transiciones

La Máquina de Turing implementada contiene 30 estados y 132 transiciones, por lo que el diagrama completo resulta excesivamente complejo para su representación detallada en el informe.

Por ello, se presenta un diagrama simplificado que agrupa los estados según su función dentro del algoritmo.

El flujo general representado en el diagrama es el siguiente:

1. Inicio del proceso.
2. Inicialización del contador n en cinta.
3. Inicialización de los bloques de trabajo a y b .
4. Verificación: ¿Quedan marcas n sin consumir?
 - Si la respuesta es **Sí**:
 - Se consume una marca ($n \rightarrow x$).
 - Se calcula $c = a + b$.
 - Se actualizan los acumuladores:

$$a \leftarrow b$$

$$b \leftarrow c$$

- Se regresa a la verificación.
- Si la respuesta es **No**:
 - Se realiza la limpieza final de símbolos auxiliares.
 - La máquina transita al estado q_{Accept} .

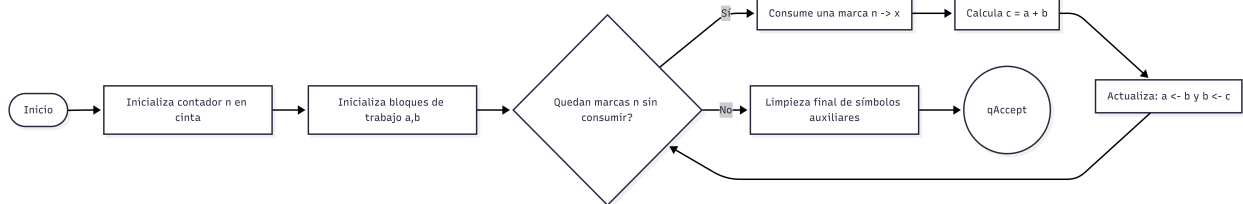


Figura 1: Diagrama simplificado del flujo funcional de la Máquina de Turing.

7. Capturas de Ejecución

A continuación se presentan capturas de la ejecución real de la Máquina de Turing implementada.

```

> python maquina_turing.py --tm fib_tm_real.json --max-n 18 --max-steps 50000000

MAQUINA DE TURING

JSON
=====
> Estado inicial: qStart
> Estado aceptor: qAccept
> Estado rechazador: qReject
> Alfabeto entrada: {1}
> Alfabeto cinta: {#, 1, A, B, _, a, b, c, n, x, |}
> Transiciones: 132
> Archivo de la MT: fib_tm_real.json
  
```

Figura 2: Inicialización de la Máquina de Turing y carga del archivo JSON.

ANALISIS EMPIRICO					
ENTRADAS DE PRUEBA Y RESULTADOS					
n	F(n) esperado	F(n) en cinta	Pasos	Tiempo (s)	Estado de aceptación
0	0	0	11	0.000014	OK
1	1	1	53	0.000048	OK
2	1	1	130	0.000070	OK
3	2	2	262	0.000129	OK
4	3	3	512	0.000251	OK
5	5	5	1009	0.000509	OK
6	8	8	2077	0.001009	OK
7	13	13	4503	0.002173	OK
8	21	21	10284	0.005038	OK
9	34	34	24512	0.011779	OK
10	55	55	60330	0.029131	OK
11	89	89	151811	0.073541	OK
12	144	144	387647	0.185117	OK
13	233	233	999177	0.503977	OK
14	377	377	2590694	1.266678	OK
15	610	610	6742026	3.267365	OK
16	987	987	17585636	8.502928	OK
17	1597	1597	45934629	22.281731	OK
18	2584	7078	50000000	24.341875	ERROR

Figura 3: Ejecución con entrada de prueba y tabla de resultados parciales.

```

MODELOS DE REGRESION POLINOMIAL
=====

Pasos de ejecucion:
> Grado del polinomio: 3
> R^2 (bondad de ajuste): 0.860499
> Ecuacion: pasos(n) = 4.64e+04*n^3 - 9.228e+05*n^2 + 4.602e+06*n - 3.813e+06
> Notacion asintotica: O(n^3)

Tiempo de CPU:
> Grado del polinomio: 3
> R^2 (bondad de ajuste): 0.860154
> Ecuacion: tiempo(n) = 0.02249*n^3 - 0.4471*n^2 + 2.23*n - 1.848
> Notacion asintotica: O(n^3)

```

Figura 4: Análisis empírico generado por el programa.

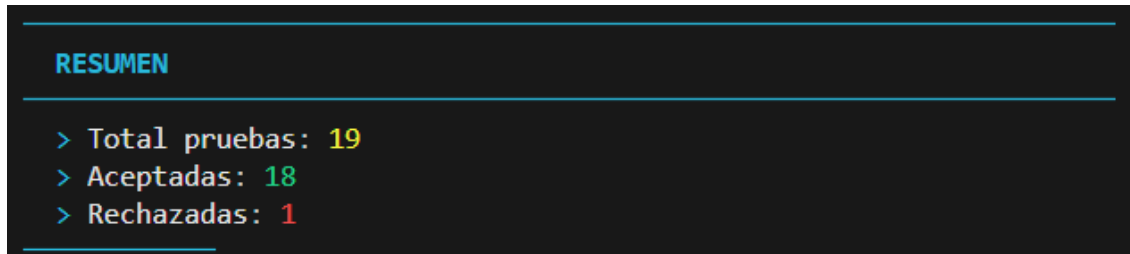


Figura 5: Resumen final de ejecuciones y métricas de rendimiento.

8. Regresión Polinomial

8.1. Pasos de ejecución

Modelo de grado 3:

$$pasos(n) = 4,64 \times 10^4 n^3 - 9,228 \times 10^5 n^2 + 4,602 \times 10^6 n - 3,813 \times 10^6$$

$$R^2 = 0,8605$$

8.2. Tiempo de CPU

Modelo de grado 3:

$$tiempo(n) = 0,02249n^3 - 0,4471n^2 + 2,23n - 1,848$$

$$R^2 = 0,8602$$

9. Notación Asintótica Experimental

Empíricamente:

$$T(n) \in O(n^3)$$

en el rango evaluado.

10. Conclusiones

La implementación actual calcula Fibonacci mediante recurrencia iterativa real en una sola cinta. El análisis empírico muestra un crecimiento aproximadamente cúbico tanto en pasos como en tiempo de CPU dentro del rango evaluado.

11. Repositorio

`https://github.com/miafuentes30/Proyecto-1-ADP.git`